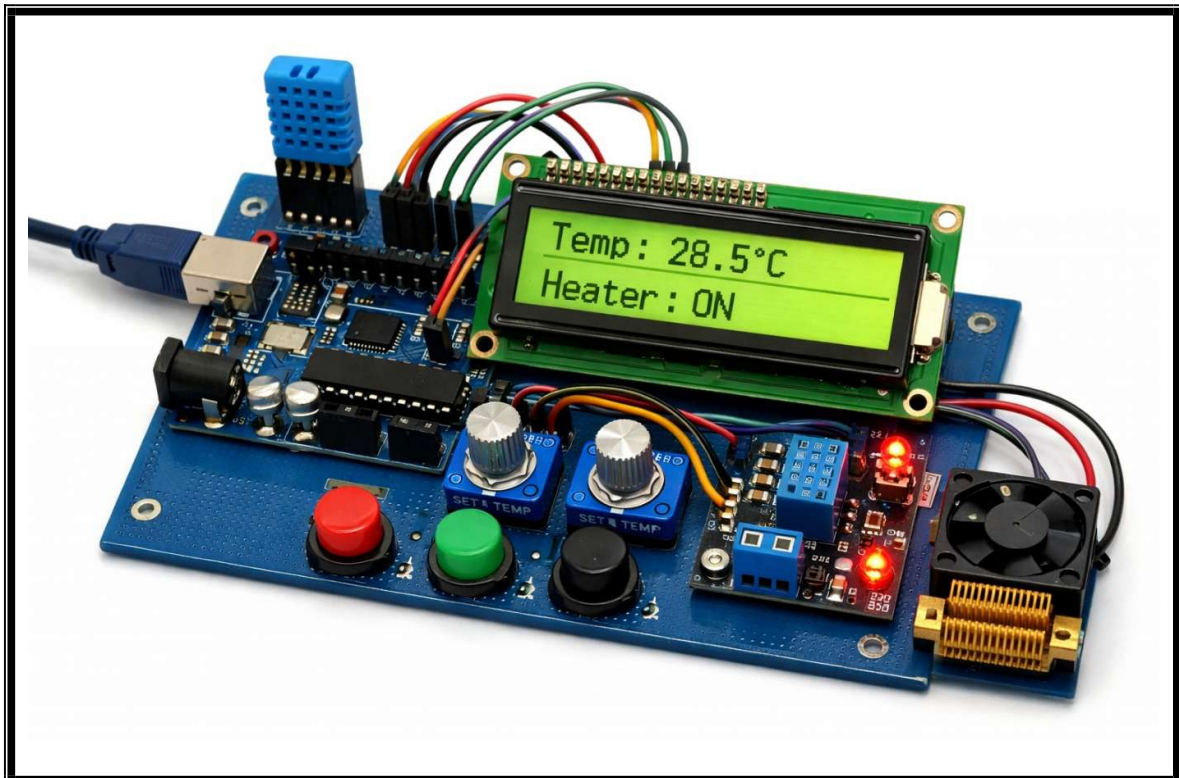


Arduino-Based Temperature Controller with DHT11, LCD, Potentiometers and Push Buttons

Prepared by:

Sr. No.	Name	Enrollment Number
1	Aditya Chaudhary	240280117001
2	Ansari Juned Ahmed	240280117002
3	Ansari Nizamuddin Vahajuddin	240280117003

Date: 20th April 2026



Abstract

This project presents a real-time temperature and humidity monitoring system with an adjustable control logic.

An Arduino Nano reads data from a DHT11 sensor, displays it on an I2C 16×2 LCD, and provides visual feedback using three LEDs (heating, stable, cooling). The user can set the target temperature (setpoint) and hysteresis via two potentiometers, toggle between Display Mode and Parameter Mode using a push button, and switch the temperature unit (Celsius, Fahrenheit, Kelvin) with another push button. A power LED blinks on button presses and sensor errors.

The system is suitable for applications such as greenhouse monitoring, server enclosures, and educational demonstrations. The code uses non-blocking timing, simple debouncing, and `map()` for linear potentiometer scaling.

1. Introduction

Maintaining a stable temperature is critical in many environments – from living spaces to industrial enclosures. This project aims to build a low-cost, intuitive temperature controller that not only displays ambient conditions but also gives clear visual cues for heating or cooling needs.

The system relies on the widely available DHT11 sensor and an Arduino Nano as the brain. Two potentiometers let the user adjust the desired temperature and the acceptable deadband (hysteresis) without reprogramming. Two buttons control the display mode and temperature unit. The design is modular and can be extended to control actual heaters or fans via relays.

Unlike commercial thermostats that often require complex menus, our system uses simple potentiometers and push buttons to set the target temperature and hysteresis. The visual LED indicators (heating, stable, cooling) give immediate, glanceable status. The design is easily extensible – with relay modules it can directly control fans or heaters – making it suitable for greenhouses, server enclosures, and educational demonstrations.

2. Components Required

Component	Quantity	Specifications / Notes
Arduino Nano	1	ATmega328P, 5V logic
DHT11 sensor	1	Temperature 0-50°C, humidity 20-90%
I2C LCD 16×2	1	Address 0x27 or 0x3F, with backpack
Potentiometers (10kΩ)	2	Linear taper; for setpoint and hysteresis
Push buttons (4-pin)	2	Momentary, normally open
LEDs (Red, White, Blue, any colour)	4	Red = heating, White = stable, Blue = cooling, Power = indicator
Resistors (330Ω)	4	Current limiting for LEDs
Breadboard & jumper wires	–	For prototyping

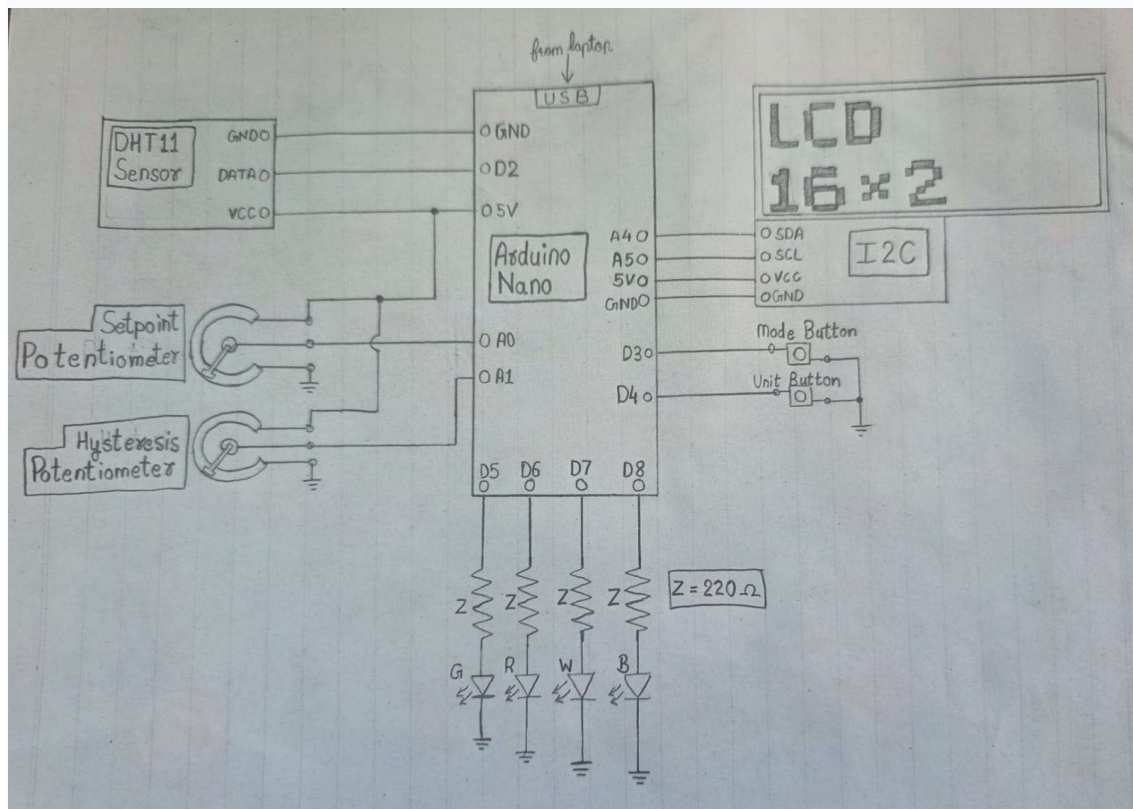
3. Circuit Diagram

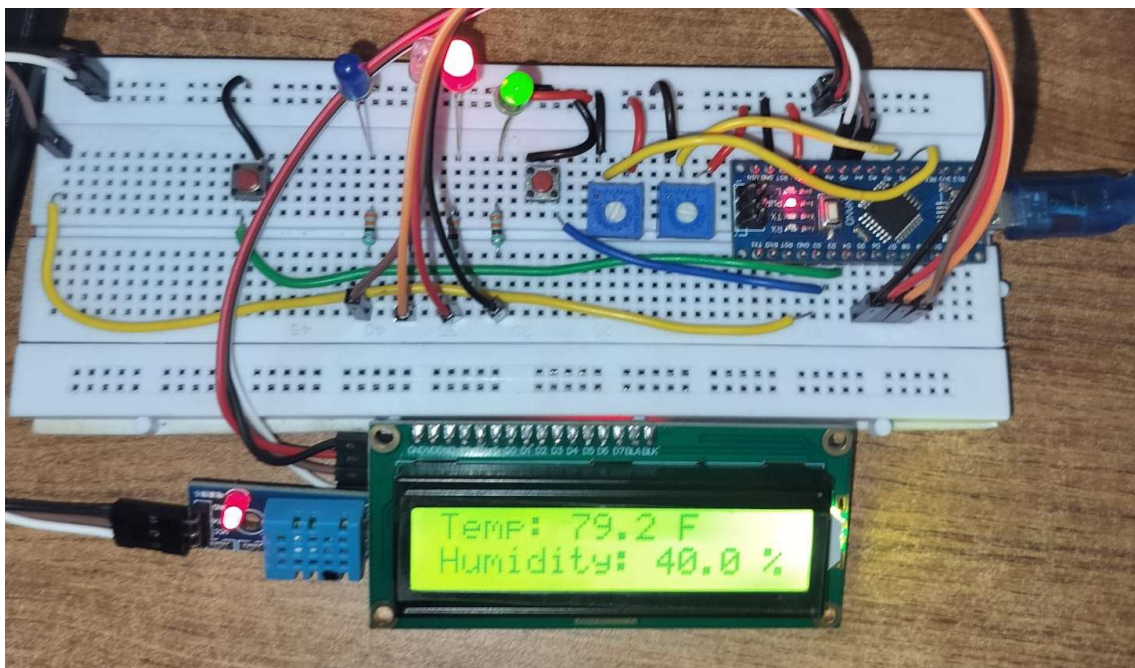
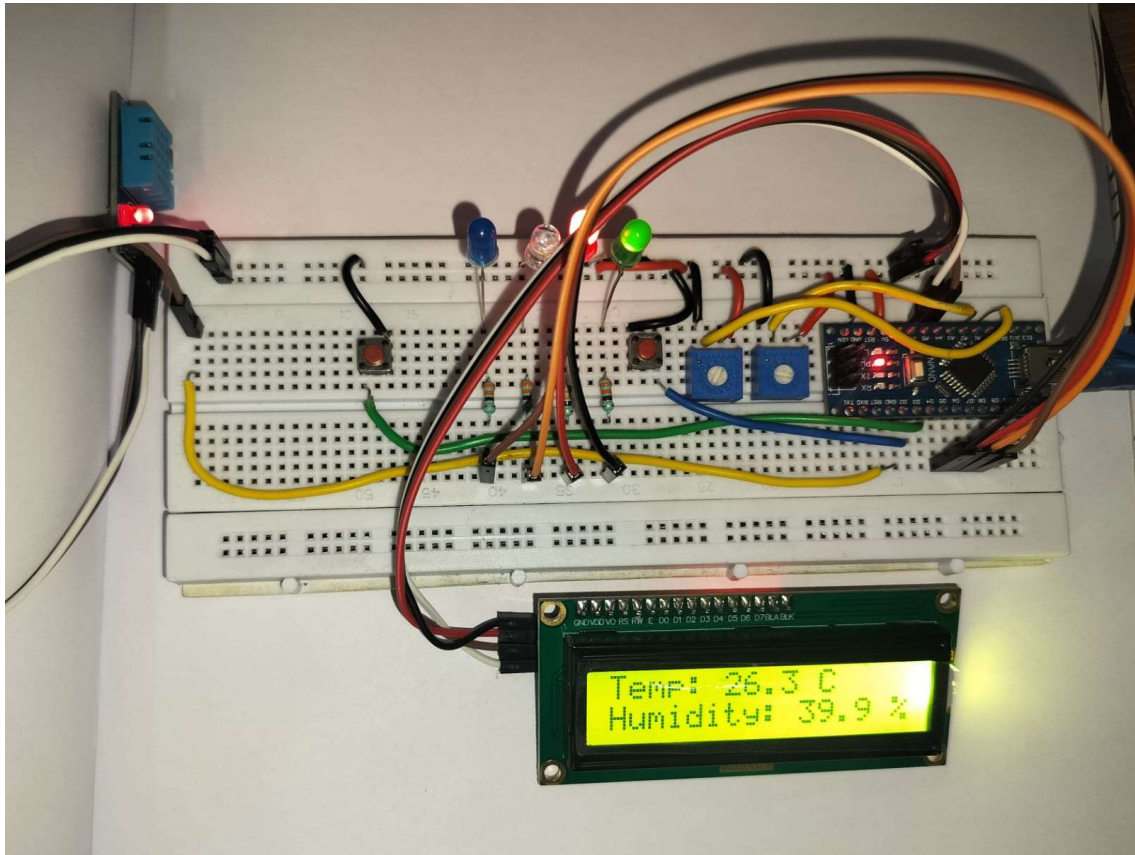
The following pin connections were used (Arduino Nano to components).

Arduino Nano Pin	Connected to
D2	DHT11 data pin
A0	Wiper of setpoint potentiometer (outer legs to 5V and GND)
A1	Wiper of hysteresis potentiometer (outer legs to 5V and GND)
D3	Mode button (other leg to GND)
D4	Unit button (other leg to GND)
D5	Power LED (anode via 220 Ω resistor, cathode to GND)
D6	Red LED (heating) + resistor
D7	White LED (stable) + resistor
D8	Blue LED (cooling) + resistor

A4 (SDA)	LCD SDA
A5 (SCL)	LCD SCL
GND	Common ground for all components

[Figure 1: Schematic diagram – see Appendix B for detailed drawing]





4. Working Principle

4.1 System Modes

The firmware implements two distinct modes: **Display Mode** and **Parameter Mode**. A short press on the Mode button (D3) toggles between them.

In Display Mode, the LCD shows the current temperature (converted to the selected unit) and humidity; the Unit button (D4) cycles through Celsius, Fahrenheit, and Kelvin.

In Parameter Mode, the LCD shows the setpoint (target temperature) and hysteresis (deadband half-width). Turning the potentiometers updates these values in real time; the Unit button has no effect in this mode.

4.2 Sensor Reading and Error Handling

The DHT11 sensor reads every 2 seconds using a non-blocking `millis()` timer. If a reading fails (e.g., `isnan()`), an error counter is incremented. After 3 consecutive failures, the LCD shows "Sensor Error!" for one second, but the system continues to use the last valid readings. The power LED blinks during the error. Once the sensor recovers, normal operation resumes automatically.

4.3 LED Indication Logic

The three status LEDs (red, white, blue) are driven by comparing the measured temperature against the setpoint $\pm$ hysteresis:

- **Red LED (heating)** → turns ON when `currentTemp < setpoint - hysteresis`.
- **Blue LED (cooling)** → turns ON when `currentTemp > setpoint + hysteresis`.
- **White LED (stable)** → turns ON when the temperature is within the desired range.

This provides an intuitive visual indication of whether the environment needs heating, cooling, or is already comfortable.

4.4 User Input (Buttons)

Both buttons are connected with internal pull-up resistors. A simple debouncing method (50 ms delay + waiting for release) ensures one reliable action per press. The power LED blinks for 100 ms on each button press to give tactile feedback.

5. Code Explanation

The main features of the code are:

- **Libraries:** Wire, LiquidCrystal_I2C, DHT sensor library (Adafruit).
- **Pin definitions and constants:** All pins are defined at the top. The potentiometer ranges are set to 20-40 °C for setpoint and 0.5-5.0 °C for hysteresis. A calibration offset (`TEMP_OFFSET`) can be adjusted to correct sensor inaccuracies.
- **Global variables:** `currentTempC`, `setpoint`, `hysteresis`, `paramMode`, `unitIndex`, etc.
- **Helper functions:** `updateLCD()` refreshes the display according to the current mode, `updateControlLEDs()` updates the red/white/blue LEDs, `convertTemperature()` handles unit conversion, and `blinkPowerLED()` gives visual feedback.
- **Setup():** Initialises pins, I2C, LCD, DHT11, and reads the initial potentiometer values so the system starts with the physical knob positions.
- **Loop():** Implements button detection, potentiometer reading (only in Parameter Mode), sensor reading every 2 seconds, and continuous LED updates.

The use of `map()` with multiplication by 100 preserves one decimal place for the setpoint and hysteresis values.

6. Testing & Results

Each subsystem was tested separately:

- **I2C LCD:** An I2C scanner sketch confirmed the address as 0x27. The LCD displayed text correctly.
- **Buttons:** A simple Serial Monitor test showed that each button press produced a single “pressed” message. The power LED blinked accordingly.
- **Potentiometers:** With the system in Parameter Mode, turning the setpoint pot changed the displayed value from 20.0 °C to 40.0 °C linearly. The hysteresis pot varied from 0.5 °C to 5.0 °C.
- **Sensor & LEDs:** Warming the DHT11 with a finger caused the measured temperature to rise; when it exceeded the setpoint + hysteresis, the blue (cooling) LED illuminated. Cooling the sensor (e.g., with a brief spray of compressed air) turned on the red (heating) LED. At room temperature, the white (stable) LED was lit.
- **Mode switching:** Pressing the Mode button toggled the LCD between Display Mode (showing temperature/humidity) and Parameter Mode (showing setpoint/hysteresis). The Unit button only worked in Display Mode, cycling C → F → K correctly.

All functionalities worked as expected. The system remained responsive and did not freeze even when the DHT11 was temporarily disconnected.

7. Calibration and Adjustments

The DHT11 has a typical accuracy of $\pm 2^{\circ}\text{C}$. To improve precision, a software offset (`TEMP_OFFSET`) was added. The offset value can be determined by comparing the sensor reading with a trusted thermometer and then adjusting the constant in the code.

The setpoint and hysteresis ranges can be easily changed by modifying the constants `MIN_SETPOINT`, `MAX_SETPOINT`, `MIN_HYST`, and `MAX_HYST`. The LCD I2C address can be set to `0x27` or `0x3F` depending on the module.

8. Applications and Future Enhancements

Applications: This device can be used for greenhouse climate monitoring, small incubators, server rack temperature alerts, room temperature indicators, and as a teaching tool for embedded systems.

Future enhancements:

- Replace DHT11 with DHT22 or BME280 for higher accuracy and wider range.
- Add relay modules to control actual heaters, fans, or air conditioners.
- Record when temperature goes outside desired range (when the heater/cooler is activated), that is implement data logging with an SD card module and a real-time clock (DS3231).
- Upgrade LCD to a more capable display such as 'OLED 128X64'
- Add a buzzer for audible alarms when temperature goes outside critical limits.
- Design a custom PCB, as the project sufficiently works on breadboard as proof of concept

9. Conclusion

A functional temperature and humidity controller was successfully designed and implemented using an Arduino Nano.

The system provides two intuitive operating modes, adjustable setpoint and hysteresis via potentiometers, and clear LED status indicators. Push buttons allow the user to change temperature units and switch between modes without needing a computer. The simple debouncing method and non-blocking sensor reading ensure reliable operation. This project demonstrates fundamental embedded concepts and can serve as a foundation for more advanced climate control systems.

10. References

- Adafruit DHT sensor library – <https://github.com/adafruit/DHT-sensor-library>
- LiquidCrystal I2C library by Frank de Brabander – [GitHub](#)
- Arduino Nano datasheet – [Arduino official documentation](#)
- DHT11 datasheet – Aosong Electronics